Here are some datasets

Problem	Classification target	Why it works in a workshop
Banknote authentication	Genuine or counterfeit	Small, numerical dataset; a good first binary classifier.
Wine cultivar	One of three wine varieties	Simple multiclass task using chemical measurements.
Dry bean classification	One of seven bean varieties	A larger multiclass problem using measurements such as area and shape.
Mushroom edibility	Edible or poisonous	Categorical features make encoding part of the exercise.
Adult income classification	Income above or below a threshold	Mixes numerical and categorical features; useful for discussing preprocessing and fairness.
Customer churn	Customer leaves or stays	A familiar applied problem with mixed features and potential class imbalance.

Here is how to fetch them

```
# pip install ucimlrepo
from ucimlrepo import fetch_ucirepo
UCI_DATASETS = {
   "banknote_authentication": 267,
   "wine": 109,
   "mushroom": 73,
   "adult_income": 2,
   "dry_bean": 602,
   "iranian_churn": 563,  # UCI option for the customer-churn problem
}
datasets = {}

for name, uci_id in UCI_DATASETS.items():
   dataset = fetch_ucirepo(id=uci_id)
   X = dataset.data.features
   y = dataset.data.targets.squeeze()
   datasets[name] = {"X": X, "y": y}
   print(f"{name}: X={X.shape}, y={y.shape}")
```

Here is the structure and the learning content. For the workshop, the target is to achieve a good overall prediction/classification outcome, even though, I have told that it is not the thing that matters for my class.

1. Start with the Dry bean classification → the dataset has 13,611 records, 16 numerical features, and seven classes.
2. Start by understanding, plotting and splitting the data. Inspect a few rows and feature names → What does one row represent? What is the label?
3. Examine class counts, create train, validation, and test splits → Why use stratification? Why keep the test set for the end?
4. Scale the features and establish a majority-class baseline → Why fit the scaler on training data only?
5. Build and train the first neural network. → Trace the tensor shapes from input to output.
6. Compare validation results and investigate errors. → What does the confusion matrix reveal that accuracy does not?
7. Experiment with one model change. → Did the change improve generalisation, or only training performance?
8. Increase or decrease hidden-layer size. Remove dropout. Change the learning rate. Try creating a grid of the hyperparameters and train again → Did the hyperparameter tuning improve generalisation, or only training performance? Which hyperparameter change has the most impact?
9. Train without standardising features, then compare.
10. Compare the neural network with a random forest or XGBoost. Try to use hyperparameter tuning there also. → can this model be used for baseline?
11. Now, do all of this for another dataset and evaluate the changes and examine the differences in the model structure and impact of hyperparameters? → can you now understand the role of hyperparameters? Do you want to try a third dataset?